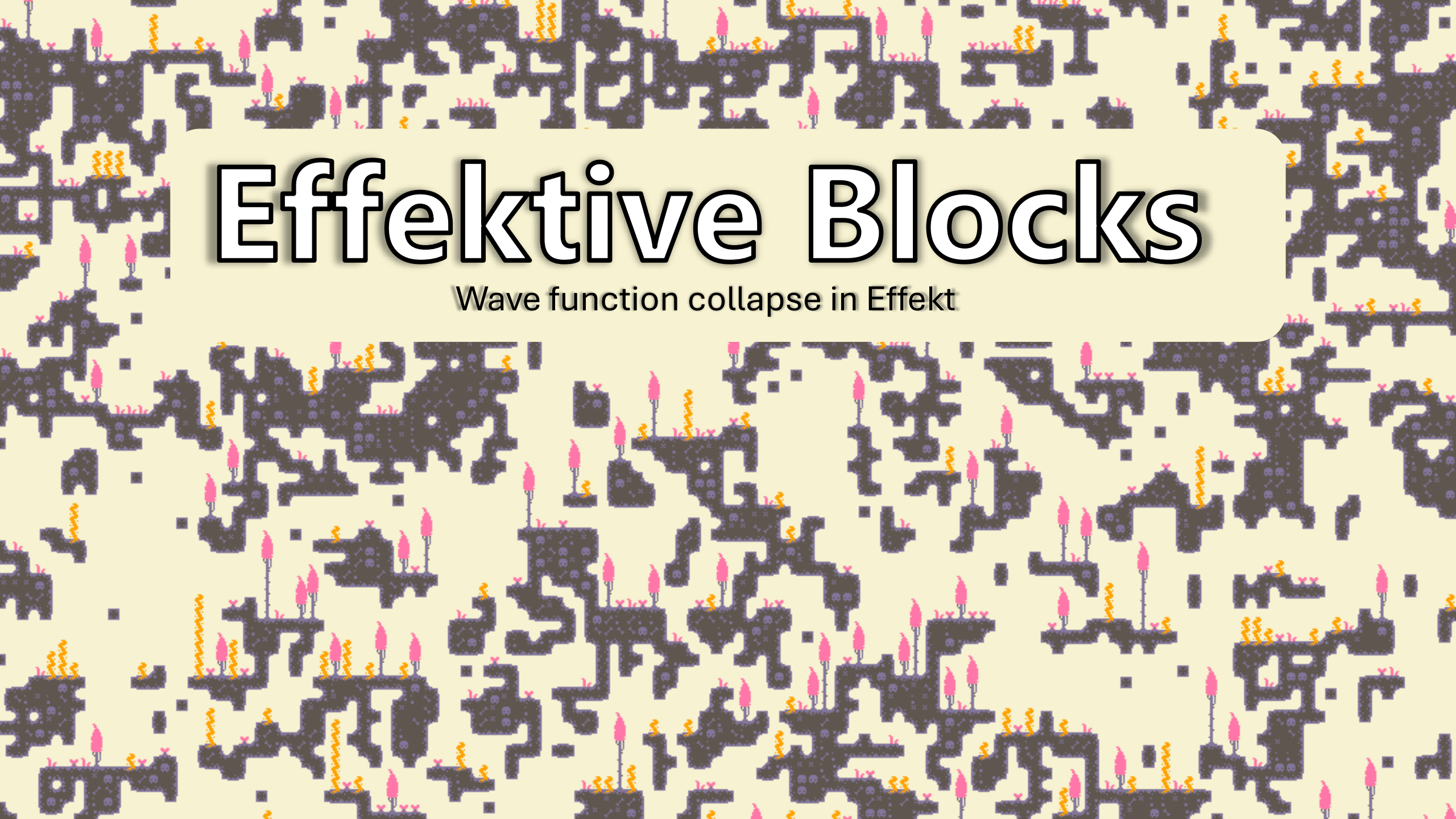
Effektive Blocks

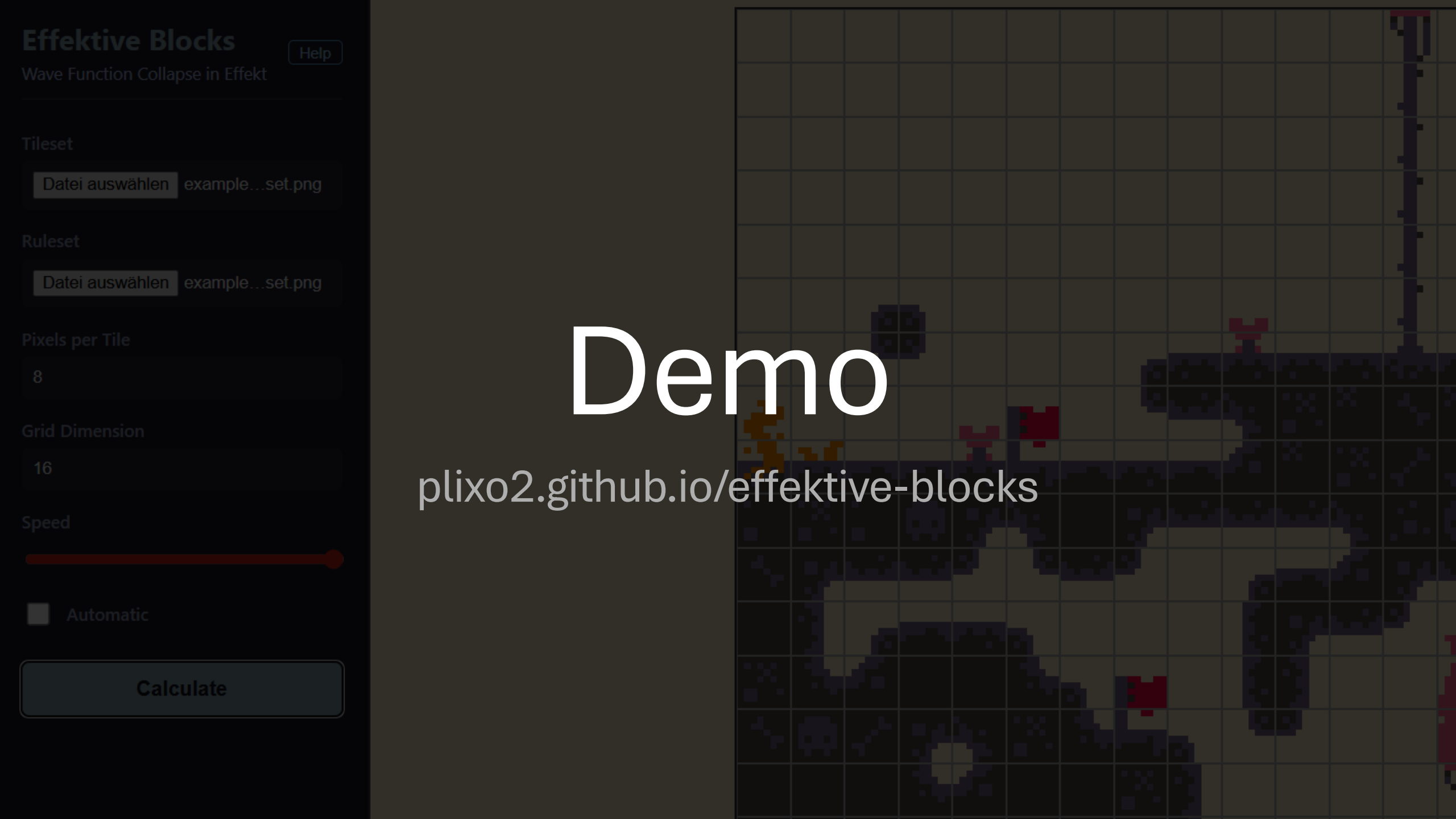
Wave function collapse in Effekt



Motivation

- Elegant Algorithm
- Explore image and data processing in Effekt
- UI’s in Effekt

```scala
def findSolution(initial: WFC::Grid, tileset: TileSet) at {io, global}: Option[WFC::Grid] = {
  val tiles = tileset.tiles
  try {
    Some(solve(initial))
  } with Search {
    def fail() = None()
    def pickTile() = {
      valueBoundary[Option[WFC::Grid]] {
        val shuffled = shuffle(tiles)
        array::foreach(shuffled) { tile =>
          resume(tile) match {
            case Some(solution) => do Break(Some(solution))
            case None() => ()
          }
        }
        None()
      }
    }
  }
}
```



Demo

plixo2.github.io/effektive-blocks

Major Effects

- Site layout
- Site styling
- User Errors
- Canvas drawing

Site layout

```
interface HTMLElement {
  def Child(child: Node): Unit
  def Text(text: String): Unit
  def parent(): Node
}
```

Site layout

```
def children[A](node: Node) { content: => A / HTMLElement } at {io}: Node = {
  try {
    val _ = content()
  }  with HTMLElement {
    def Child(child: Node) = {
      FFI::Dom::appendChild(node, child)
      resume(())
    }
    def Text(content: String) = {
      val text = FFI::Dom::createTextNode(content)
      FFI::Dom::appendChild(node, text)
      resume(())
    }
    def parent() = resume(node)
  }
  node
```

Site layout

```
with body.onChildren  // called once per application
column {
  row {
    h2("Effektive Blocks")
  }
}
```

Site *styling*

```
effect Attribute(key: String, value: String): Unit at {io}
def attributes(node: Node) { scope: => Unit / Attribute }: Node = {
  try {
    scope()
  } with Attribute { (key, value) =>
      FFI::Dom::setAttribute(node, key, value)
      resume(())
  }
  node
}
```

Site *styling*

```
div {
  label(name).css {
    property("font-size", "0.85rem")
    property("font-weight", "600")
    property("color", "#a6adc8")
  }.attributes {
    attribute("for", name)
  }
}
```

Popup Effect

```
val color: Image = color?.getOrElse {
  do Popup(s"Provide a color image")
}
val mask: Image = mask?.getOrElse {
  do Popup(s"Provide a mask image")
}
```

Other Effekt features

- Uniform function call syntax (UFCS)

```
def popup[E, T](proxy: on[E]) { prog: => T / Exception[E] }: T / Popup
def tileAt(grid: TileGrid, x: Int, y: Int): Tile / Popup = {
  with on[OutOfBounds].popup

  grid.get(x).get(y)
}
```

Other Effekt features

- String interpolation

```
def cssString(self: Color) at {global}: String = {
  colorFormat"rgba(${self.r}, ${self.g}, ${self.b}, ${self.a.toDouble})"
}

def show(self: Color) at {global}: String = {
  colorFormat"Color(${self.r}, ${self.g}, ${self.b}, ${self.a})"
}
```

Other Effekt features

- String interpolation

```
def text { str: => Unit / {literal, splice[Int], ...}}: Unit / HTMLElement = {

  do Text(stringBuffer  {
    try {
      str()
      do flush()
    }
    with literal { s => resume(do write(s)) }
    with splice[Int] { i => resume(do write(i.show))}
    ...
  })

}
```

Other Effekt features

- String interpolation

```
button() {
  text "Help"
}.css {
  property("background", "none")
```

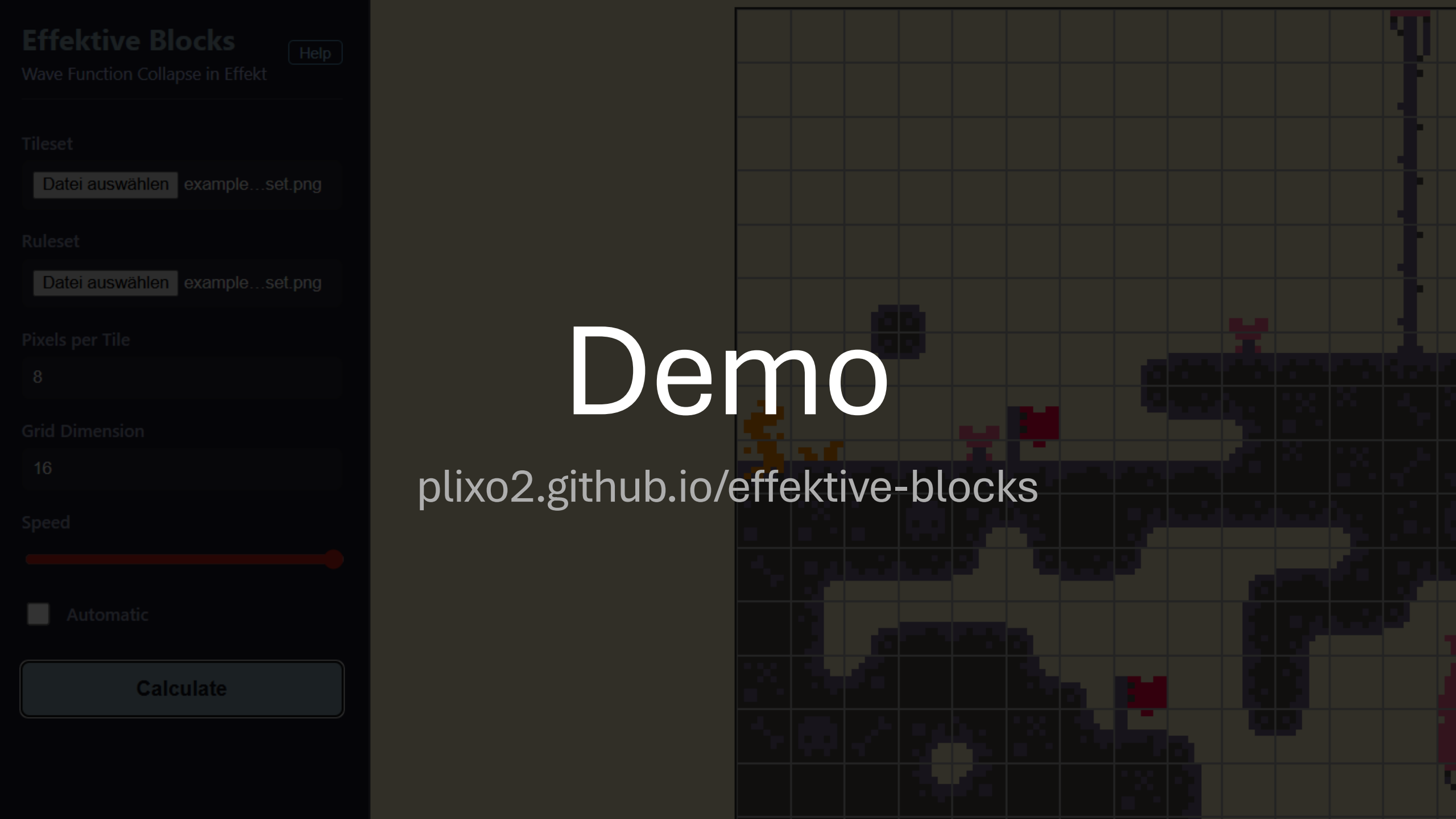
Challenges & Key decisions

- Procedural WFC algorithm
- Javascript arrays vs Effekt lists
- Big FFI, but minimal surface area and good abstractions
- Speed
- Compile times (--max-inline-size 10)

Q&A

Credits

- https://github.com/mxgmn/WaveFunctionCollapse
- https://robertheaton.com/2018/12/17/wavefunction-collapse-algorithm/
- https://bolddunkley.itch.io/wfc-mixed / https://www.youtube.com/watch?v=2SuvO4Gi7uY
- https://oskarstalberg.com/game/wave/wave.html
- https://kenney.nl (Example assets)



Demo

plixo2.github.io/effektive-blocks